

Ejercicio 1: Análisis y Diseño

Tabla de propiedades

#	Tipo de datos	Nombre	Visibilidad	Motivo	Clase
1	cliente	Cliente	Private	Obtener y guardar datos del cliente.	ControladorBoletos
2	vistaEntradas	vista	Private	Mandar a llamar a la clase vistaEntradas.	ControladorBoletos
3	Localidad	localidad1	Private	Guardar la información de localidad1.	ControladorBoletos
4	Localidad	localidad5	Private	Guardar la información de localidad5.	ControladorBoletos
5	Localidad	localidad10	Private	Guardar la información de localidad10.	ControladorBoletos
6	Scanner	sc	Private	Permitir que el cliente ingrese los datos requeridos.	VistaBoletos
7	String	nombre	Private	Guardar el nombre del cliente.	Cliente
8	String	email	Private	Correo electrónico del cliente.	Cliente
9	Int	cantidadBoletos	Private	Registrar la cantidad de boletos deseados.	Cliente
10	Float	presupuestoMax	Private	Registrar el precio que está dispuesto a pagar.	Cliente

1 1	Int	numeroTicket	Private	Generación del número aleatorio para asignar al ticket.	Ticket
1 2	Int	limiteA	Private	Designar un límite inferior/superior para pasar a la siguiente fase.	Ticket
1 3	Int	limiteB	Private	Designar el límite inferior/superior para pasar a la siguiente fase.	Ticket
1 4	Int	cantBoletos	Private	Designar una cantidad de boletos por localidad.	Localidad
1 5	String	nombre	Private	Darle nombre a la localidad.	Localidad
1 6	Int	precioBoleto	Private	Dar el valor del boleto de acuerdo con la localidad (100, 500 y 1000).	Localidad
1 7	int	boletosVendidos	Private	Almacena la cantidad de boletos que han sido vendidos.	Localidad
1 8	Float	caja	Private	Conocer los datos acumulados de las ventas realizadas por localidad.	Localidad

Tabla de métodos

#	Tipo de retorno	Nombre	Visibilidad	Motivo	Parámetros	Clase
1	void	main	Public	Tener un método principal en la clase principal	Args: String[]	Main
2	void	iniciar	Public	Iniciar el programa	-	Controlador Boletos
3	void	nuevoComprador	Privat e	Ingresar los datos de un nuevo comprador	-	Controlador Boletos
4	void	nuevaSolicitud	Privat e	Iniciar una nueva solicitud de compra	-	Controlador Boletos
5	Void	consultaDisponibilidadTotal	Privat e	Consultar la disponibilidad total de boletos en las tres localidades	-	Controlador Boletos
6	void	consultaDisponibilidadIndividual	Privat e	Consultar la disponibilidad por localidad	-	Controlador Boletos
7	void	reporteCaja	Privat e	Reporte de los boletos vendidos en cada localidad	-	Controlador Boletos

				y las ganancias		
8	void	Salir	Privat e	Terminar el programa	-	Controlador Boletos
9	void	AsignarLocalidad	Privat e	Asignar una localidad aleatoria al cliente cuando el ticket es válido. Utilizado en la nueva solicitud.	-	Controlador Boletos
10	int	mostrarMenu	Privat e	Darle al usuario opciones a elegir en el programa.	-	VistaBoletos
11	Cli ente	leerDatosCliente	Public	Almacena r los datos del comprador.	-	VistaBoletos
12	void	mostrarFaltaCliente	Public	Mostrar que sin cliente no puede continuar.	-	VistaBoletos
13	Void	mostrarTicketRechazado	Public	Mostrar que el número de ticket fue rechazado .	-	VistaBoletos
14	int	leerLocalidad	public	Leer la disponibilidad de la localidad	-	VistaBoletos

				que quiere ver el usuario.		
15	void	mostrarDisponibilidadTotal	Public	Mostrar la disponibilidad de los boletos según la localidad.	localidad1: Localidad, localidad5: Localidad, localidad10; Localidad	VistaBoletos
16	void	mostrarDisponibilidadIndividual	Public	Determinar si aún hay espacio o ya se vendieron los 20 boletos por localidad.	localidad: Localidad	VistaBoletos
17	void	mostrarLocalidadLlena	Public	Mostrar cuando una localidad ya no tiene boletos.	-	VistaBoletos
18	void	mostrarPresupuestoInsuficiente	Public	Mostrar cuando el presupuesto del cliente es insuficiente.	-	VistaBoletos
19	void	mostrarCompra	Public	Mostrar mensaje de que la compra fue realizada.	-	VistaBoletos
20	void	mostrarReporteCaja	Public	Si se cumplen todas las consideraciones	localidad1: Localidad, localidad5: Localidad,	VistaBoletos

				anteriores , se mostrará la venta del boleto a la caja y la suma de los ingresos.	localidad10; Localidad	
21		Cliente	Public	Registrar los datos del cliente	nombre: String, email: String, cantidadBoleto: int, presupuestoMax:float	Cliente
22	String	getNombre	Public	Obtener el nombre	-	Cliente
23	String	getEmail	Public	Obtener email	-	Cliente
24	Int	getCantidadBoletos	Public	Obtener la cantidad de boletos deseados	-	Cliente
25	Float	getPresupuestoMax	Public	Obtener el presupuesto máximo del cliente	-	Cliente
26		Ticket	Public	Crear un nuevo ticket	-	Ticket
27	void	crearTicket	Public	Generar un número de ticket entre 0 a 15000	-	Ticket
28	void	crearLimites	Public	Generar límites inferior y superior de 0 a 15000.	-	Ticket

29	Bool	IsValid	Public	Llevar a cabo la validación del ticket con sus límites para poder avanzar en la compra.	-	Ticket
30		Localidad	Public	Asignar el precio dependiendo de la localidad por instancia	Nombre: String, precioBoleto : float	Localidad
31	float	getPrecioBoleto	Public	Obtener el precio del boleto	-	Localidad
32	int	getBoletosVendidos	Public	Obtener la cantidad de boletos vendidos	-	Localidad
33	float	getCaja	Public	Obtener el dinero que se ha ganado en la localidad	-	Localidad
34	Bool	validarPresupuesto	Private	Validar si el cliente cuenta con el dinero suficiente para efectuar la compra	presupuesto Max: Float	Localidad
35	int	espaciosDisponibles	Public	Calcular si aún existen espacios	-	Localidad

				disponibles en las localidades.		
36	int	boletosDisponibles	Public	Verificar la existencia de boletos deseados, de no existir espacio, vender la cantidad disponible sin exceder los boletos deseados.	cantidadBoletos: Int	Localidad
37	void	agregarVentas	Public	De poder realizar la compra, se agrega la venta a caja, sumando la cantidad.	boletosComprados: int	Localidad

¿Cómo clasifica cada una de sus clases según el patrón MVC (Model, View, Controller)? Justifique brevemente el rol que cumple cada una dentro de esa clasificación.

Cada una de las clases según el patrón se divide de manera que el Model se divide en Cliente, el cual almacena sus datos para dar seguimiento a su solicitud de compra; Ticket, el cual genera números aleatorios para este junto con límites inferior y superior según la forma de venta diseñada; y Localidad, el cual realiza las distintas operaciones dentro para verificar si se puede efectuar la compra en distintas localidades.

En cambio, en View se encuentra VistaBoletos, la cual permite la interacción programa-usuario, logrando que el programa pueda ser utilizado por este.

Además, en Controller se encuentra el ControladorBoletos el cerebro del programa que despliega el menú, logrando recibir información y unir las clases de vista y modelo.

Fuera de este se encuentra la clase Main, la cual representa el punto de partida del programa.

¿Qué ventaja ofrece separar la lógica de negocio (Model) de la interacción con el usuario (View/Controller) en este sistema? ¿Qué problema evita esta separación si en el futuro se necesita modificar el algoritmo de asignación de tickets?

Según el patrón MVC, el rol que cumple cada una es, el model serían la clase que llevan los datos y variables a utilizar en las demás, siendo en sí los datos a utilizar. View es la clase que se mostrará y con la que podrá interactuar el usuario. Por último, Controller lleva a cabo distintas operaciones que controlan las demás clases, técnicamente podría decirse que es el cerebro de todo el programa, ya que es el punto de conexión entre las clases View y Model.

En este sistema separar la lógica del negocio es beneficioso para que, al momento de modificar el algoritmo de asignación de Tickets, no se tenga que modificar el programa completamente y menos que se interfiera la clase con la que interactúa el usuario (Vista). Es decir, esta separación permite que se pueda modificar la parte deseada sin afectar el funcionamiento de las demás.

Diagrama UML

